

1.Import Libraries

```
!pip install pandas
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

2.Load the Dataset

```
df = pd.read_csv('/content/netflix_titles.csv')
df.head(2)
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...

3.Basic Dataset Info

```
df.shape
```

```
(8807, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   show_id         8807 non-null   object
1   type            8807 non-null   object
2   title           8807 non-null   object
3   director        6173 non-null   object
4   cast            7982 non-null   object
5   country         7976 non-null   object
6   date_added      8797 non-null   object
7   release_year    8807 non-null   int64
8   rating          8803 non-null   object
9   duration        8804 non-null   object
10  listed_in       8807 non-null   object
11  description      8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

```
df.describe(include='all')
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
count	8807	8807	8807	6173	7982	7976	8797	8807.000000	8803	8804	8807	8807
unique	8807	2	8807	4528	7692	748	1767	NaN	17	220	514	8775
top	s8807	Movie	Zubaan	Rajiv Chilaka	David Attenborough	United States	January 1, 2020	NaN	TV-MA	1 Season	Dramas, International Movies	Paranormal activity at a lush, abandoned prope...
freq	1	6131	1	19	19	2818	109	NaN	3207	1793	362	4
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2014.180198	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN	8.819312	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1925.000000	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2013.000000	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2017.000000	NaN	NaN	NaN	NaN

4.Checking Missing Values

df.isnull().sum()	
	0
show_id	0
type	0
title	0
director	2634
cast	825
country	831
date_added	10
release_year	0
rating	4
duration	3
listed_in	0
description	0
dtype: int64	

5.Handle Missing Values

<pre>df['director'].fillna('Unknown',inplace=True) df['cast'].fillna('Unknown',inplace=True) df['country'].fillna('Unknown',inplace=True) df['date_added'].fillna('Unknown',inplace=True) df['rating'].fillna('Unknown',inplace=True) df['duration'].fillna('Unknown',inplace=True)</pre>
/tmp/ipython-input-3439976809.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me <pre>df['director'].fillna('Unknown',inplace=True) /tmp/ipython-input-3439976809.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu</pre> For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me <pre>df['cast'].fillna('Unknown',inplace=True) /tmp/ipython-input-3439976809.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu</pre> For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me <pre>df['country'].fillna('Unknown',inplace=True) /tmp/ipython-input-3439976809.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu</pre>

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me

df['date_added'].fillna('Unknown',inplace=True)
/tmp/ipython-input-3439976809.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me

df['rating'].fillna('Unknown',inplace=True)
/tmp/ipython-input-3439976809.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assign
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting valu

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me

df['duration'].fillna('Unknown',inplace=True)
```

6.Convert Data Types

```
df['date_added'] = df['date_added'].replace("Unknown", np.nan)
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
```

```
df['date_added'] = df['date_added'].replace("Unknown", pd.NA)
```

```
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
```

```
df['date_added'] = df['date_added'].replace("Unknown", pd.NA)
```

```
df['date_added'] = df['date_added'].fillna("Unknown")
```

```
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
```

Completely remove rows with missing data

```
df = df.dropna(subset=['date_added'])
```

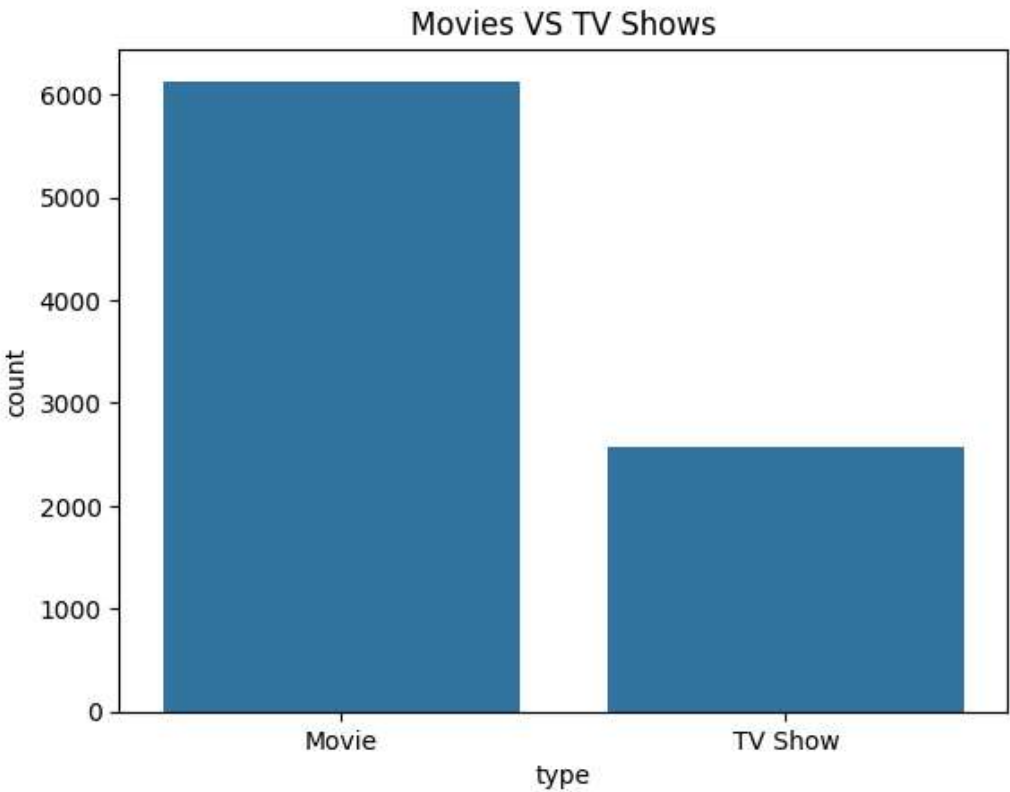
```
df.isnull().sum()
```

	0
show_id	0
type	0
title	0
director	0
cast	0
country	0
date_added	0
release_year	0
rating	0
duration	0
listed_in	0
description	0
dtype:	int64

7.Analysis & Visualizations

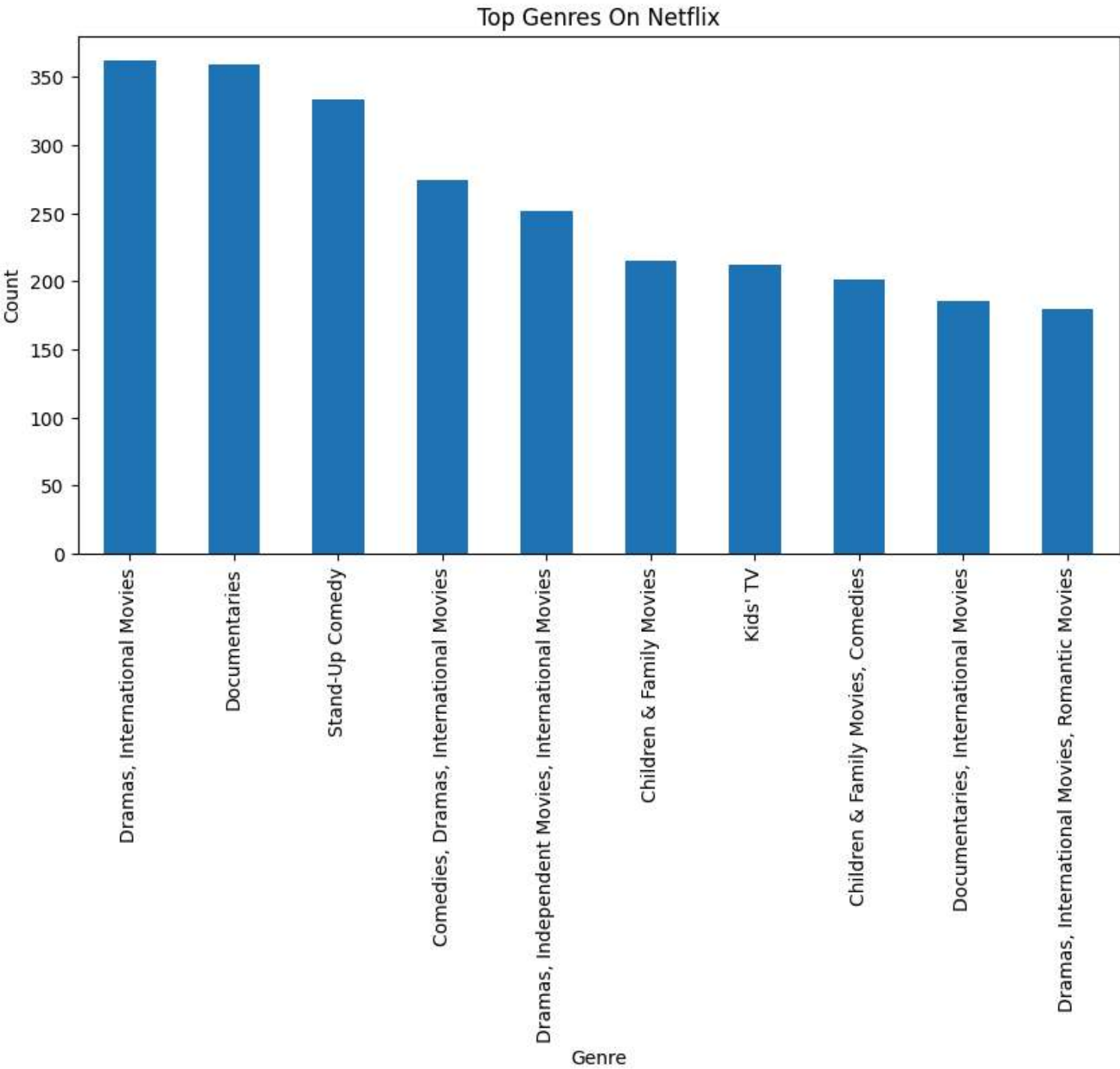
Count of Movies vs TV Shows

```
sns.countplot(x='type', data=df)
plt.title('Movies VS TV Shows')
plt.show()
```



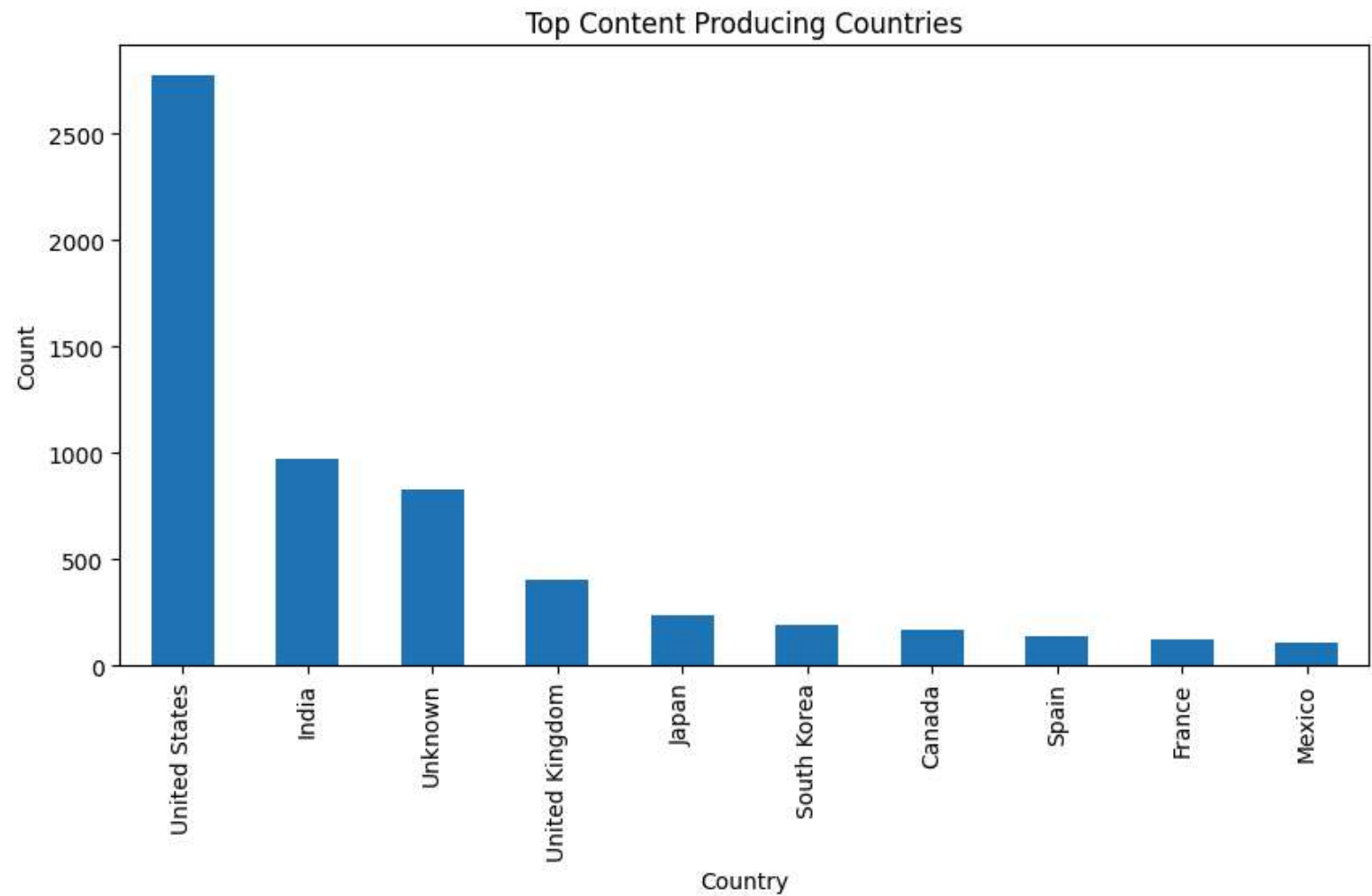
Most Common Genres

```
plt.figure(figsize=(10, 5))
df['listed_in'].value_counts().head(10).plot(kind='bar')
plt.title('Top Genres On Netflix')
plt.xlabel('Genre')
plt.ylabel('Count')
plt.show()
```



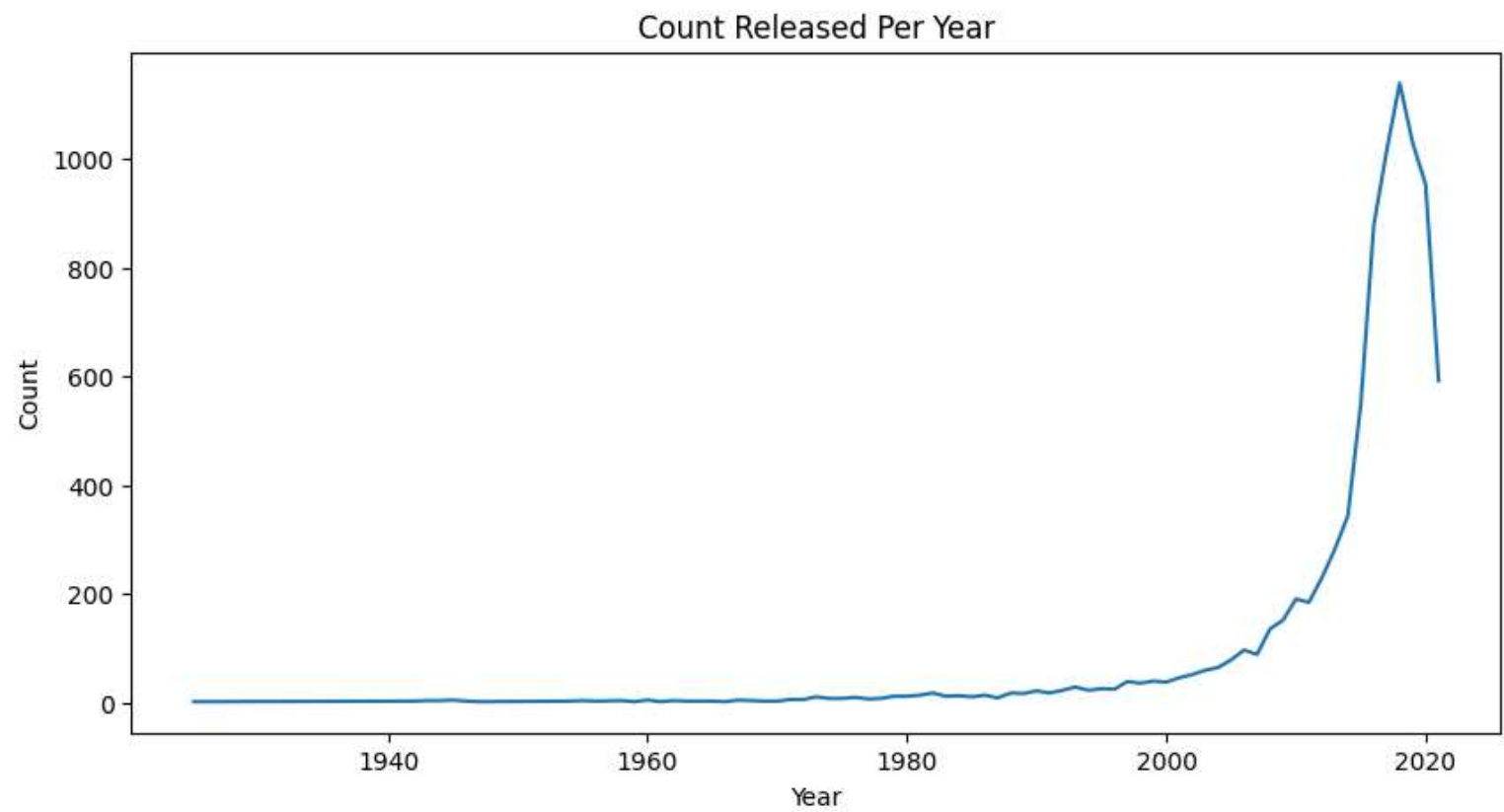
Top 10 Countries Producing Netflix Content

```
plt.figure(figsize=(10, 5))
df['country'].value_counts().head(10).plot(kind='bar')
plt.title('Top 10 Countries Producing Netflix Content')
plt.xlabel('Country')
plt.ylabel('Count')
plt.title('Top Content Producing Countries')
plt.show()
```



Releases By Year

```
plt.figure(figsize=(10, 5))
df.groupby('release_year').size().plot()
plt.title('Count Released Per Year')
plt.xlabel('Year')
plt.ylabel('Count')
plt.show()
```



Which Director has the Most Content ?

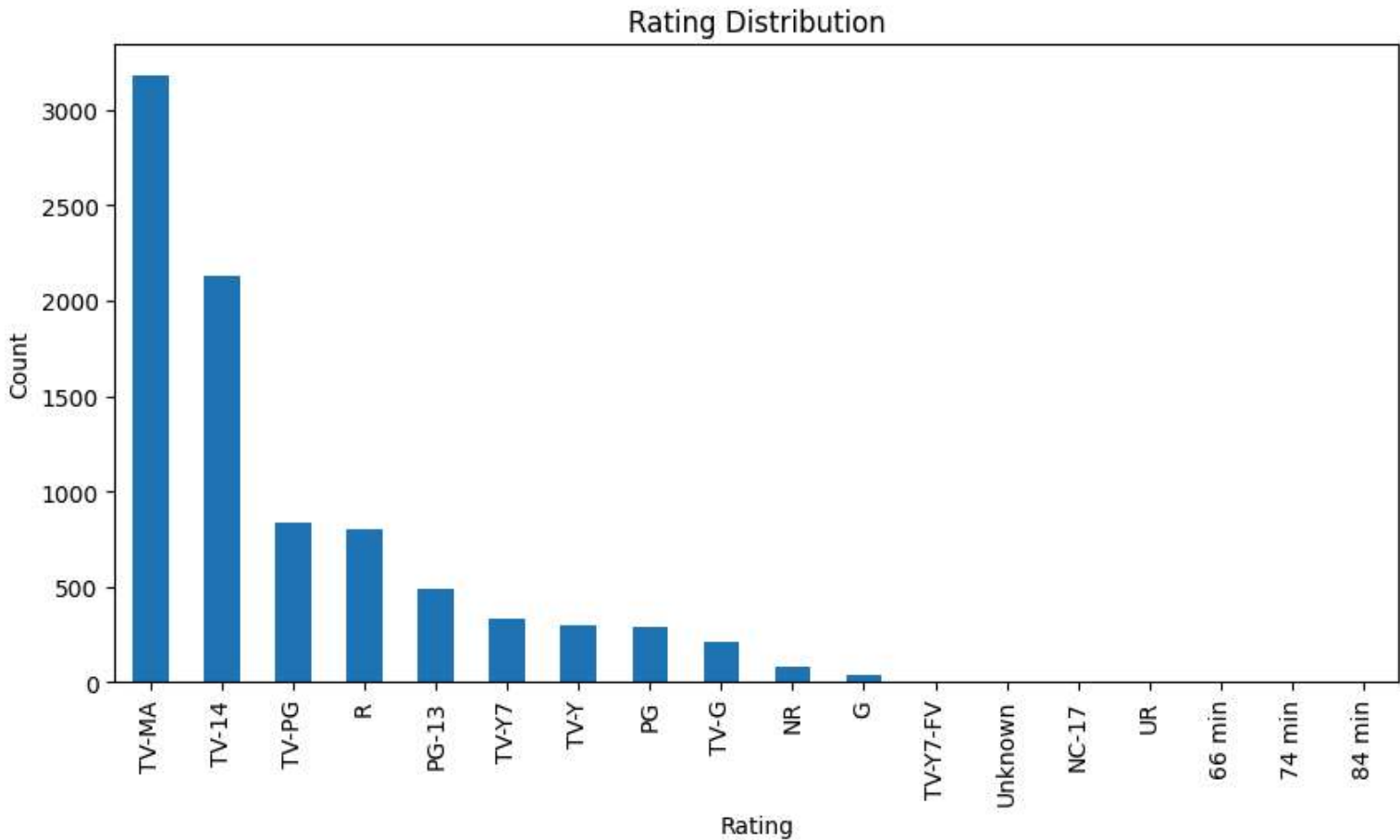
```
df['director'].value_counts().head(10)
```

director	count
Unknown	2541
Rajiv Chilaka	19
Raúl Campos, Jan Suter	18
Suhas Kadav	16
Marcus Raboy	16
Jay Karas	14
Cathy Garcia-Molina	13
Youssef Chahine	12
Martin Scorsese	12
Jay Chapman	12

dtype: int64

Most Common Ratings(TV-MA, TV-14....)

```
plt.figure(figsize=(10, 5))
df['rating'].value_counts().plot(kind='bar')
plt.title('Rating Distribution')
plt.xlabel('Rating')
plt.ylabel('Count')
plt.show()
```

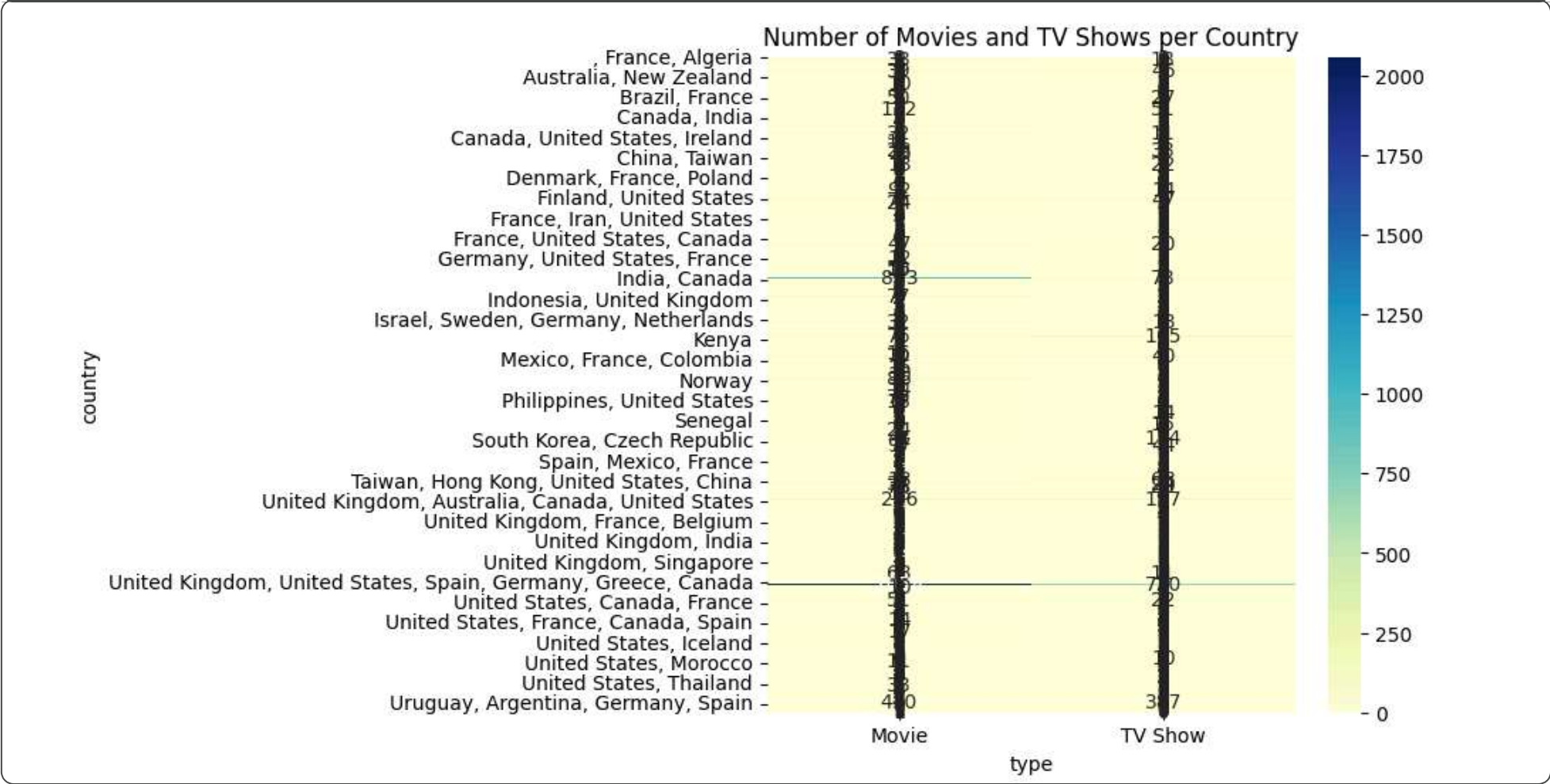


Word Cloud (Genres)

```
from wordcloud import WordCloud

plt.figure(figsize=(10, 6))
wordcloud = WordCloud(width=800, height=400, background_color='black').generate(' '.join(df['l:
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Genre Word Cloud')
plt.show()
```





- USA dominates both Movies and TV shows.
- India produces mostly Movies, smaller proprtion of TV shows.

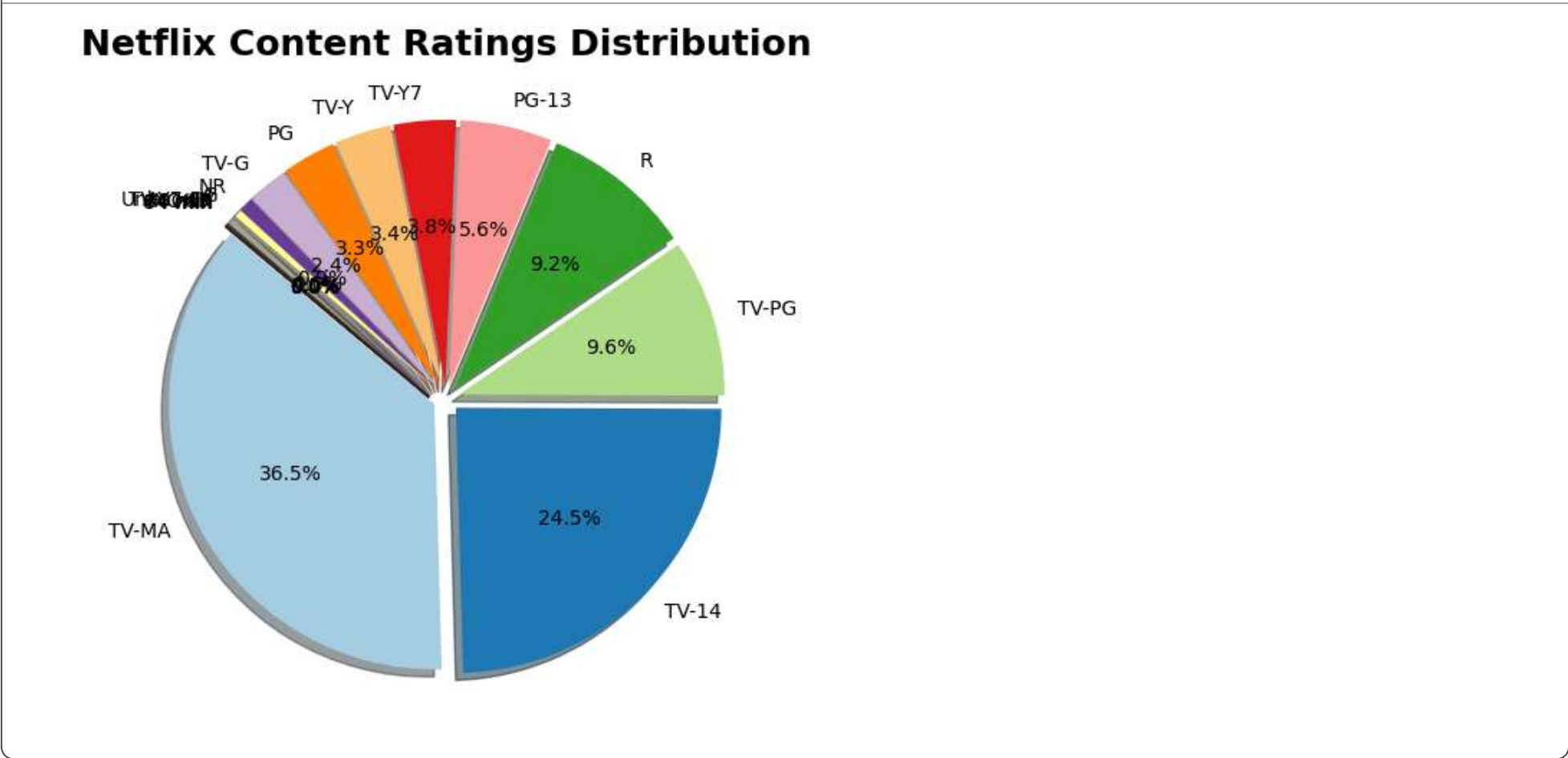
```
import matplotlib.pyplot as plt

# Count of each rating
rating_counts = df['rating'].value_counts()

# Pie chart
plt.figure(figsize=(6,6))
colors = plt.cm.Paired.colors # colorful palette

plt.pie(rating_counts, labels=rating_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, shadow=True, explode=[0.05]*len(rating_counts))

plt.title('Netflix Content Ratings Distribution', fontsize=18, fontweight='bold')
plt.show()
```



▼ Kye Insights.....